

Primera aproximación a una metodología de desarrollo de software para realidad aumentada basada en taxonomía de procesos

Quinnie Villarreal Aragón, José Manuel Carvajal, Jonathan Sandoval, Lina Ballesteros, Alejandro Ríos
Universidad EAFIT
Procesos Modernos de Desarrollo de Software
Medellín, Colombia

Abstract—Augmented reality (AR) systems combine real and virtual objects in real time with spatial registration, yet software projects in this domain lack a consolidated methodology that jointly addresses the delivery of software and content, coordination among distributed components, explicit user validation, and the social and legal obligations arising from continuous environment capture. This paper reviews the domain and its life cycle, and examines four software engineering approaches: generative artificial intelligence (GenAI), DevOps, event-driven architecture (EDA), and human-centered design (HCD). Building on the process-model taxonomy of Céret et al. and on situational method engineering, it proposes a first methodological approximation, defined over a seven-phase model and characterized along six axes: cycle, collaboration, artifacts, recommended use, maturity, and flexibility. Its distinctive elements are a two-week microcycle with fixed closing deliverables, nested between continuous delivery and a validated increment of one to three months; two mandatory return gates assigned to named roles, one for performance and one for user experience and social responsibility, plus a conditional gate for generated content; nine internal roles, including product owner, technical lead, and a rights and licensing officer who clears the provenance of every asset entering the content chain; a dual construction pipeline for code and augmentation content; mixed artifact formalization governed by an explicit boundary criterion; declared limits of applicability; and configurable variability, with HCD and DevOps mandatory and EDA and GenAI conditional. It is an unvalidated first approximation and provides instrumentation and outcome indicators as a baseline for empirical validation.

Index Terms—Augmented reality, DevOps, event-driven architecture, generative AI, human-centered design, process models, process taxonomy, software methodology

I. INTRODUCCIÓN

La realidad aumentada (RA; en inglés, *augmented reality*, AR) combina objetos reales y virtuales de forma interactiva, en tiempo real y con registro espacial tridimensional [1]. El dominio pasó de prototipos de laboratorio a despliegues móviles e industriales con madurez parcial: es viable en nichos concretos, pero persisten retos de *tracking*, interacción, evaluación con usuarios y coordinación de sistemas socio-técnicos [2], [3]. En la práctica, los equipos deben co-diseñar hardware, *pipelines* de tracking y renderizado, contenido gráfico y reglas de interacción [4], [5].

El problema no es solo técnico. Aunque existen guías de arquitectura, ciclos multimedia adaptados y marcos de

evaluación de usabilidad, sigue siendo limitada una metodología que integre de forma explícita la entrega continua de software y contenido, la reacción a eventos en experiencias distribuidas, la validación reiterada con usuarios (déficit que la evidencia identifica como propio del dominio [6]) y las obligaciones sociales y legales derivadas de capturar el entorno de forma continua [7]. Sin esa integración, las decisiones sobre ciclo, roles, artefactos, límites de uso y madurez quedan fragmentadas entre enfoques parciales.

Este artículo propone una primera aproximación metodológica para el desarrollo de software en RA. Como marco se adopta la taxonomía de modelos de proceso de Céret et al., que organiza el diseño de metodologías en seis ejes: ciclo, colaboración, artefactos, uso recomendado, madurez y flexibilidad [8]. Sobre ese andamiaje se combinan cuatro enfoques: GenIA (*inteligencia artificial generativa*), para la generación controlada de contenido cuando hay datos confiables; DevOps, para entrega repetible de software y contenido; EDA, para experiencias distribuidas y en tiempo real; y HCD, para elevar la evaluación con usuarios a fase de primera clase. La propuesta se formula sobre un modelo de siete fases y define un microciclo de dos semanas con entregables fijos, dos compuertas de retorno obligatorias más una condicional, nueve roles internos y dos externos, doble cadena de construcción, formalización mixta y variabilidad configurable. La comparación se plantea frente a Spiral, RAD (*Rapid Application Development*) y XP (*Extreme Programming*), según las clasificaciones publicadas en [8].

II. DOMINIO: ESTADO ACTUAL DE LA REALIDAD AUMENTADA

II-A. Definición y estado de desarrollo

La RA se define clásicamente como un sistema que combina objetos reales y virtuales en un entorno real, opera de forma interactiva y en tiempo real, y registra ambos en tres dimensiones [1]. La definición es agnóstica al dispositivo y no se restringe al visor montado en la cabeza ni al sentido de la vista [2]. Milgram y Kishino la sitúan en el *continuum* realidad-virtualidad, donde el entorno predominante es el mundo real enriquecido con contenido digital [9].

Azuma identificó tempranamente el registro, la latencia y la sensorización como problemas centrales [1], sin resol-

verse la alineación estable en entornos no controlados [2]. Goh et al. observan que la investigación reciente se orienta a adopción masiva, ética y accesibilidad, además de retos persistentes de display, tracking e interacción [3]; revisiones técnicas sistematizan tracking, herramientas, RA colaborativa y preocupaciones de seguridad y privacidad [7], y Devagiri et al. ofrecen un panorama de hardware, software y guías de desarrollo [5]. El estado es de *madurez parcial*: viable comercialmente en móviles y nichos industriales y educativos, con la adopción generalizada aún abierta [3], [5].

II-B. Abordaje de proyectos, arquitectura y ciclo de vida

Los proyectos de RA se abordan como sistemas socio-técnicos donde hardware, software y contenido se co-diseñan [4], [5]. Cuatro decisiones estructuran el abordaje: la delimitación del contexto de uso, incluidos usuarios, entorno físico y restricciones de seguridad [10], [11]; la selección de plataforma y modalidad de display [7]; la estrategia de anclaje espacial mediante marcadores fiduciales, SLAM o localización visual-inercial [7]; y el diseño de la interfaz espacial, atendiendo a carga cognitiva y coherencia entre lo real y lo virtual [12]. Arquitectónicamente, los sistemas se descomponen en captura, tracking y mapeo, modelo del entorno, lógica de aplicación, renderizado e interacción, con requisitos transversales de tiempo real; la especificación ETSI GS ARF 003 formaliza esa descomposición como arquitectura funcional de referencia con despliegue en dispositivo, nube o híbrido [13].

De la convergencia entre el *Multimedia Development Life Cycle* adaptado a RA [14], las guías contemporáneas [5] y el marco de HCD y evaluación de usabilidad [10], [15] resulta un ciclo de siete etapas: encuadre y contexto; requisitos funcionales y no funcionales (latencia, precisión de registro, autonomía, privacidad); diseño arquitectónico; preparación de activos y prototipado; implementación del *pipeline* de tracking, render e *input*; evaluación técnica y de usabilidad; y despliegue, operación y evolución. Ese ciclo se toma como estructura base en la Sección IV (Cuadro II).

Dos rasgos condicionan el coste del proyecto y explican la selección de enfoques. El primero es que la preparación de activos constituye una etapa propia, con producción y mantenimiento diferenciados, y no un anexo de la implementación: los marcos específicos la tratan como fase separada [14] y las guías contemporáneas la incluyen entre las capacidades que un equipo debe cubrir [5]. El segundo es que, aunque la evaluación con usuarios figura como fase explícita en los marcos revisados [10], [15], la evidencia sobre su adopción efectiva y sobre métricas propias del dominio es limitada [6].

II-C. Metodologías utilizadas

No existe una única metodología estándar universal para RA; la literatura converge en la combinación de marcos de desarrollo multimedia/software con diseño centrado en el humano (HCD) y evaluación específica de usabilidad en RA [5], [14], [15].

III. ENFOQUES SELECCIONADOS Y RAZÓN DE ELECCIÓN

Los cuatro enfoques se seleccionaron con tres criterios derivados de la Sección II. El primero es cubrir una debilidad documentada: entrega conjunta de software y contenido, coordinación entre componentes distribuidos, coste de autoría de activos y escasez de validación con usuarios. El segundo es disponer de literatura o normalización que permita caracterizar el enfoque con las gradaciones de [8]. El tercero es la complementariedad entendida como especialización: los enfoques coinciden en varias fases, pero cada uno aporta la decisión dominante en al menos un eje distinto (Cuadro I). Quedaron fuera candidatos pertinentes que se señalan como extensión natural: desarrollo dirigido por modelos para generación de contenido 3D, MLOps para el ciclo de los modelos de visión que sostienen el tracking, e ingeniería de seguridad y privacidad como disciplina completa de gestión de amenazas, relevante por la captura continua de entorno [7]; esta metodología cubre la privacidad únicamente como requisito verificable en compuerta (Sección IV-H), no como esa disciplina completa. El conjunto es el mínimo que cubre las cuatro debilidades identificadas, no una lista exhaustiva.

III-A. Inteligencia artificial generativa (GenIA)

GenIA emplea modelos que generan contenido nuevo a partir de patrones aprendidos de grandes volúmenes de datos. En ingeniería de software apoya requisitos, diseño, programación, pruebas, documentación y mantenimiento. Los proyectos se ejecutan de forma iterativa y controlada: identificar el problema, preparar y validar datos, diseñar, construir un producto mínimo viable, evaluar y desplegar con monitoreo; para información actualizada se propone RAG (*Retrieval-Augmented Generation*), que recupera de fuentes autorizadas antes de generar [16]. Las técnicas son *prompt engineering*, RAG, *human-in-the-loop*, generación estructurada, *guardrails*, evaluación automática y humana, y versionamiento de *prompts*; las herramientas, servicios de IA, *frameworks* de integración, bases de datos vectoriales, Git, Docker y observabilidad. No exige lenguaje de modelado propio: se proponen UML, BPMN, C4 Model, OpenAPI y JSON Schema. Los métodos complementarios son Agile, DevOps, MLOps, DevSecOps, TDD y gestión de riesgos de IA, para la que se toma como referencia el perfil de IA generativa del marco de NIST [17].

Se elige por su vínculo con un cuello de botella del dominio: el coste de autoría del contenido. Una aplicación de RA no despliega solo código, sino activos 3D, anotaciones espaciales y reglas de interacción que deben producirse y mantenerse para cada escenario; como esa producción es una etapa propia con coste diferenciado [5], [14], reducir el esfuerzo de autoría es el vínculo más directo con la RA, y la literatura reciente explora esa aplicación en generación multimodal para narrativa aumentada [18]. Como referente de ejecución se toma la práctica industrial observada en Genius Sports, descrita solo a nivel de prácticas: plataformas que combinan grandes volúmenes de datos deportivos con aprendizaje automático para análisis, personalización y augmentación de transmisiones. Por acuerdo de confidencialidad no se divulgan nombres de servicios

internos, rutas ni configuraciones, y las herramientas citadas son alternativas de la propuesta, no tecnologías confirmadas del referente.

III-B. DevOps

DevOps busca acortar el tiempo entre un cambio y su operación en producción manteniendo calidad, mediante colaboración entre desarrollo y operaciones, automatización y entrega continua [19], [20]. En ingeniería de software articula integración y entrega continuas (CI/CD), infraestructura como código y monitoreo, prácticas que condicionan arquitectura y operación [19]. Los proyectos se ejecutan por promoción escalonada: cada cambio dispara el *pipeline*, el resultado se empaqueta como candidato de versión, se valida en ambientes intermedios con firma de los roles implicados y solo entonces se promueve a producción. Las herramientas pertenecen a cinco familias, con alternativas de uso extendido: motores de integración continua (Jenkins, GitLab CI, GitHub Actions), contenerización y registros de imágenes (Docker), orquestación (Kubernetes con Helm), provisionamiento declarativo y gestión de secretos (Terraform, Ansible, HashiCorp Vault) y observabilidad (Prometheus, Grafana). Los lenguajes son declarativos: *pipelines* en YAML, infraestructura como código y esquemas JSON para los artefactos de contenido. Los métodos son la entrega continua por candidatos de versión, las convenciones de registro de cambios que permiten derivar la versión automáticamente y el aseguramiento de calidad operacional previo a producción.

Se elige porque el dominio exige iterar con seguridad operativa sobre software y contenido a la vez. Como referente se toma la práctica observada en *augmentación gráfica* en tiempo real sobre video deportivo, donde junto al *pipeline* de código opera un *pipeline* de contenido que produce instrucciones de *augmentación*. Ese referente es un reporte de observación profesional, no reproducible por terceros ni recogido con protocolo de investigación, por lo que ilustra una configuración posible y no constituye evidencia; el elemento que de él se incorpora, la separación entre cadena de código y cadena de contenido, se triangula con literatura que distingue la producción de contenido como etapa propia [5], [14]. Se declara el matiz de que el referente es *augmentación* sobre video *broadcast* y no RA móvil con visor, aunque comparte la necesidad de entregar software y contenido gráfico de forma repetible. Como en la Sección III-A, no se divulgan detalles internos.

III-C. Arquitectura orientada a eventos (EDA)

EDA es un estilo arquitectónico en el que componentes desacoplados producen, reciben y procesan eventos de forma asíncrona bajo un modelo productor–consumidor; el desacoplamiento favorece modularidad, escalabilidad e integración de sistemas distribuidos [21]. En ingeniería de software afecta diseño, calidad, despliegue y observabilidad, y se combina con microservicios, *domain-driven design* y CQRS, pero su adopción debe evaluarse por contexto: la evidencia empírica muestra que no garantiza mejores tiempos de respuesta ni

menor consumo de recursos frente a un monolito [21]. Los proyectos se ejecutan iterativamente: identificar eventos del dominio y sus productores y consumidores, definir contratos y reglas de procesamiento, implementar el flujo, validar funcionalidad, latencia, *throughput* y resiliencia, desplegar y monitorear. Las técnicas son publicación/suscripción, *event streaming* y procesamiento de eventos complejos, con herramientas como Apache Kafka, RabbitMQ y motores CEP. No posee lenguaje único: se proponen diagramas UML de secuencia y componentes y contratos AsyncAPI en JSON o YAML, que describen APIs orientadas a mensajes de forma legible por máquina y agnóstica al protocolo [22]. Los métodos son el diseño orientado a contratos de eventos y la validación iterativa por pruebas de integración, rendimiento y resiliencia. Se elige porque las experiencias RA distribuidas y colaborativas requieren reaccionar a cambios de interacción, sesión y estado compartido: SARA demuestra una arquitectura RA colaborativa donde servicios y clientes intercambian eventos para actualizar sesiones y elementos aumentados [23].

III-D. Diseño centrado en el humano (HCD)

HCD busca hacer usables y útiles los sistemas interactivos centrándose en los usuarios, sus necesidades y el contexto de uso. A diferencia de los demás enfoques está normalizado: ISO 9241-210 fija seis principios, entre ellos la participación del usuario durante todo el desarrollo, la iteración y el equipo multidisciplinario, y cuatro actividades cíclicas que operan como secuencia de ejecución y no como declaración de principios: entender el contexto de uso, especificar requisitos, producir soluciones de diseño y evaluar contra los requisitos, de modo que el resultado de la evaluación decide si el proyecto avanza o retorna a cualquiera de las tres anteriores [15]. Su relación con la ingeniería de software se articula en la *Human-Centered Software Engineering*, que integra la usabilidad en el ciclo de vida en lugar de tratarla como fase cosmética final [24], convirtiéndola en requisitos no funcionales medibles y elevando la evaluación a fase con capacidad de retorno [25]. Los proyectos no siguen un proceso único sino una familia de modelos (Star Life Cycle, Usage-Centered Design, Usability Engineering Lifecycle) comparados por Ferre et al. [25].

Las técnicas se distribuyen por actividad: entrevistas contextuales y observación para el contexto; personas y escenarios para los requisitos; prototipado de fidelidad creciente y *Wizard of Oz* para el diseño; y pruebas de usabilidad, evaluación heurística [26] e instrumentos como la *System Usability Scale* (SUS) o el NASA-TLX para la evaluación. Las herramientas son repositorios de hallazgos y mapeo colaborativo, editores interactivos como Figma o Adobe XD, motores como Unity con componentes de realidad mixta cuando la interacción es espacial, y plataformas de prueba de usabilidad remota. En modelado dispone de notaciones formales, como Concur-TaskTrees para tareas [27], aunque la práctica se apoya en artefactos semi-formales (personas, mapas de recorrido, *storyboards*); esa asimetría es relevante para el eje de artefactos. Se elige porque hay evidencia de brecha entre lo que ofrece y lo que el dominio aplica: una revisión sistemática de 30

artículos filtrados de 498 no identificó métricas de experiencia específicas de RA [6], y la investigación en RA ha desatendido la dimensión de usuario [28].

IV. ESPINA DE PESCADO: TAXONOMÍA DE PROCESOS Y PROPUESTA METODOLÓGICA

Los seis ejes que estructuran esta sección constituyen la *espina de pescado* de la propuesta: cada uno recoge el aporte de los cuatro enfoques (Cuadro I) y converge en la definición del proceso. La Fig. 1 muestra la elaboración del equipo sobre esos ejes durante el diseño; las decisiones finales se fijan en las subsecciones siguientes. Conviene precisar el alcance de la taxonomía de [8]: es un esquema de *clasificación*, no de *definición*. Por ello la Sección IV-A antepone un modelo de fases explícito y los ejes posteriores fijan las decisiones sobre él. El enfoque general corresponde a la ingeniería de métodos situacional, que construye métodos a partir de componentes configurables según el contexto [29].

IV-A. Modelo de fases, microciclos y compuertas

La propuesta se proyecta sobre el ciclo de vida de la Sección II-B, que se toma como estructura base y se detalla en el Cuadro II.

Compuertas. Se emplea el término *compuerta de retorno* en lugar de *puerta de calidad* porque el elemento definido no solo autoriza o bloquea el avance, sino que prescribe la fase de destino del retroceso según la causa registrada. La compuerta de rendimiento (G1) verifica latencia, *throughput* y resiliencia sobre el flujo de datos y el renderizado; la ejecuta el rol de calidad de integración y rendimiento, y es obligatoria con independencia de la activación de EDA, porque la latencia de registro y renderizado es una restricción inherente a la RA [1] y no una consecuencia de la comunicación asíncrona. La compuerta de experiencia y responsabilidad social (G2) verifica los requisitos de uso mediante evaluación con usuarios y las obligaciones de la Sección IV-H; la ejecutan el especialista en experiencia de usuario y el responsable de derechos y licencias, con participación del usuario final. La compuerta de generación (G3) solo existe cuando GenIA está activo y verifica calidad, trazabilidad y procedencia del contenido generado; la coejecutan el responsable de contenido y el responsable de derechos y licencias, con participación del responsable de los datos; a diferencia de G1 y G2, situadas en una fase única, G3 se aplica a la salida de toda fase en que GenIA produzca contenido destinado al producto, es decir F4 y F5. Ningún contenido generado entra a la cadena de despliegue sin superarla.

Microciclo de dos semanas. Se definen tres cadencias anidadas que operan a distinta escala. La *integración y entrega técnica* es continua: cada cambio dispara el *pipeline* correspondiente y puede desplegarse automáticamente en ambientes de integración o validación. La promoción a producción ocurre cuando se han superado las compuertas aplicables: G3 para el contenido generado mediante GenIA, cuando este enfoque está activo, y G1 y G2 para declarar completo el incremento validado. El *microciclo* es la unidad de planificación y coordinación,

de dos semanas de duración fija; abre con una sesión en la que el *product owner* y el líder técnico seleccionan trabajo del *backlog* y cierra con entregables fijos y una revisión. El *incremento validado* agrupa de dos a seis microciclos, esto es de uno a tres meses, y solo se declara completo cuando G1 y G2 se han superado sobre el conjunto acumulado; lo acepta el *product owner*. Un despliegue a un ambiente de validación no constituye un microciclo, y un microciclo no constituye por sí solo un incremento validado.

El microciclo no recorre las siete fases: selecciona trabajo de las fases activas en ese momento del proyecto. F1 a F3 suelen concentrarse en los primeros microciclos y F4 a F7 adquieren mayor recurrencia posteriormente; sin embargo, cualquier fase puede reactivarse cuando una evaluación o compuerta prescriba un retorno, evitando así reproducir una cascada en miniatura. Al cierre se revisan y actualizan seis elementos de control, según corresponda a las fases activas del microciclo: (i) una compilación desplegable del incremento de código; (ii) el manifiesto de contenido de augmentación versionado, con los activos incorporados y su registro de derechos; (iii) el contrato de frontera actualizado y versionado si cambió; (iv) la evidencia de las compuertas ejecutadas cuando corresponda, con la causa registrada de cada retorno; (v) el registro de decisiones arquitectónicas; y (vi) el *backlog* repriorizado y aceptado por el *product owner*. Cuando alguno de estos elementos aún no aplique o no haya sufrido cambios durante el microciclo, esta condición se registra explícitamente en lugar de producir un artefacto artificial; así, un microciclo dedicado a F1 puede cerrar sin compilación desplegable ni manifiesto de contenido, sin que ello incumpla el principio de entregables fijos, que fija categorías de cierre y no la obligación de poblarlas todas en cada iteración. Al menos un microciclo de cada incremento validado debe incluir sesiones de evaluación con usuarios y entregar su informe, lo que impide diferir indefinidamente la validación que motiva la inclusión de HCD.

En F5 conviven dos cadenas de construcción, la de código y la de contenido. Esta *dualidad de pipelines* procede del referente de la Sección III-B y es una particularidad del dominio: en RA lo que se despliega no es solo software, sino activos, instrucciones de augmentación y reglas de interacción, con su propio versionado y sus propias pruebas.

IV-B. Ciclo

Los cuatro enfoques coinciden en la iteración pero difieren en qué la dispara: HCD retorna desde la evaluación con usuarios; EDA desde las pruebas de latencia y resiliencia; GenIA desde la evaluación previa al despliegue; DevOps aporta entrega continua con promoción por ambientes. Se propone un ciclo incremental cuya unidad básica de planificación es el microciclo de dos semanas, mientras que la integración y entrega técnica opera de forma continua sobre esa base (Sección IV-A); esa cadencia de planificación sitúa la propuesta en el grado más fino de la escala de [8], con entregas en menos de un mes. Sobre esta cadencia se define una unidad distinta, el incremento validado, que agrupa entre

Cuadro I
APORTE DE LOS CUATRO ENFOQUES A LOS SEIS EJES DE LA TAXONOMÍA

Eje	GenIA	DevOps	EDA	HCD
Ciclo	Iterativo y controlado; la evaluación precede al despliegue y puede devolver a la preparación de datos.	Entrega continua; promoción por candidatos de versión y ambientes escalonados, con excepciones documentadas.	Retorno desde las pruebas de latencia y resiliencia; productores y consumidores avanzan en paralelo una vez fijado el contrato.	Retorno explícito obligatorio a contexto, requisitos o diseño; granularidad local y regional [15].
Colaboración	Supervisión humana permanente como control, no como revisión final; exige responsable de los datos.	Calidad multi-rol antes de producción: operaciones, ingeniería y contenido firman por separado.	Coordinación por contratos de eventos compartidos, que reducen la dependencia directa entre equipos.	Usuario como rol externo formal en todo el ciclo; equipo multidisciplinario por principio normativo [15].
Artefactos	Prompts versionados, datos validados y salidas estructuradas bajo <i>guardrails</i> ; UML, BPMN, C4, OpenAPI.	Pipelines en YAML, imágenes de contenedor, infraestructura como código, esquemas JSON. Formalización alta: son ejecutables.	Catálogo y esquema de eventos, matriz productor-consumidor, AsyncAPI. Formalización media-alta [22].	Contexto de uso, personas, escenarios, prototipos e informes con métricas. Formalización mixta [27].
Uso recomendado	Con datos confiables y actualizados; exige gestionar riesgos de privacidad, sesgo y propiedad intelectual [17].	Cuando hay que entregar software y contenido de forma repetible y el sistema debe seguir disponible.	Para experiencias distribuidas o de tiempo real con múltiples productores; no en aplicaciones pequeñas y locales.	Cuando la madurez de requisitos es baja y el contexto es incierto, condición típica de la RA [6].
Madurez	Emergente; los marcos de gestión de riesgo son recientes [17].	Alta; amplia adopción industrial y literatura consolidada [20].	Media-alta en general, con menor documentación específica en RA [21].	Alta; normalizado internacionalmente y con dos décadas de literatura [24].
Flexibilidad	Alta; no exige lenguaje propio y se combina con Agile, DevOps, MLOps y TDD.	Alta; plantillas reutilizables y caminos excepcionales previstos y registrados.	Alta; admite distintos <i>brokers</i> y agregar consumidores, a costa de gobernar el versionado de contratos.	Alta variabilidad: al menos cuatro modelos de proceso bajo el mismo marco [25].

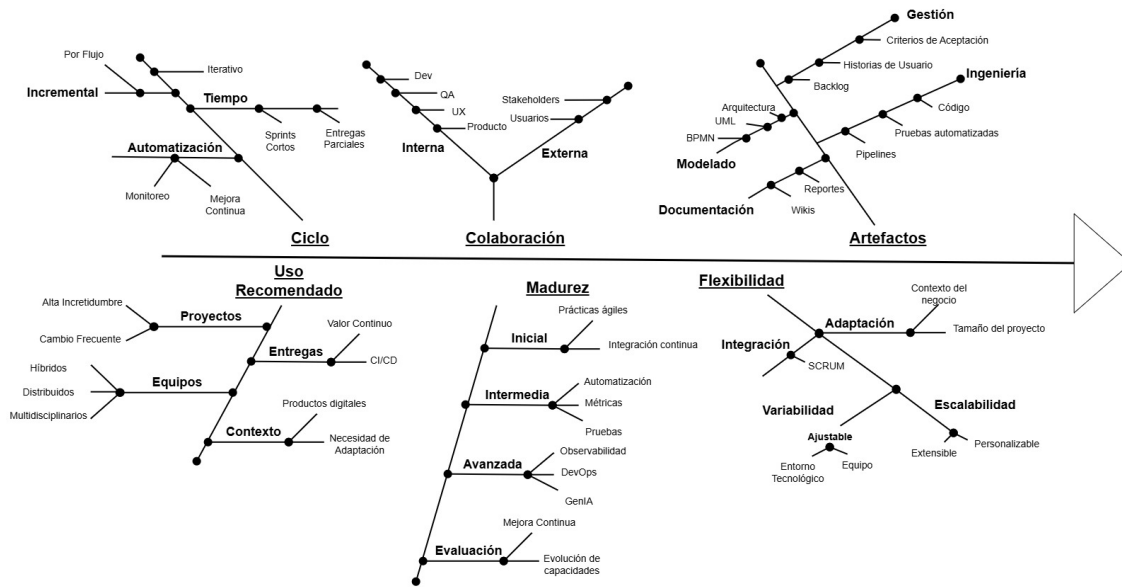


Figura 1. Espina de pescado elaborada por el equipo durante el diseño de la propuesta: los seis ejes de la taxonomía como espinas principales y los elementos aportados por los enfoques como espinas secundarias. Las decisiones finales sobre cada eje se desarrollan en las Secciones IV-B a IV-G. La responsabilidad social (IV-H) se trata como una obligación transversal adicional, verificada en compuerta, y no forma parte de los seis ejes de la taxonomía.

dos y seis microciclos y tiene una duración de uno a tres meses; esa capa adicional no está contemplada en la escala de [8]. La iteración es local sobre el flujo de eventos y el prototipo de interacción, y regional sobre el conjunto de experiencia y datos. El retorno es explícito y obligatorio a través de las compuertas, lo que responde a una carencia concreta: RAD menciona procedimientos de validación pero no especifica qué hacer cuando una etapa no se valida [8].

IV-C. Colaboración y roles

HCD incorpora al usuario como rol externo formal; DevOps aporta calidad multi-rol; EDA coordina mediante contratos compartidos; y GenIA exige supervisión humana permanente. Sobre esa base se definen nueve roles internos con responsabilidades diferenciadas. El *product owner* prioriza el *backlog*, selecciona el trabajo de cada microciclo y acepta los incrementos validados. El líder técnico responde por las decisiones arquitectónicas y los compromisos entre rendimiento, calidad

Cuadro II
MODELO DE FASES, ENFOQUES ACTIVOS, SALIDAS Y COMPUERTAS

Fase	Enfoques activos	Salida principal	Compuerta
F1. Encuadre y contexto de uso	HCD	Especificación del contexto de uso; alcance acordado	Sin compuerta
F2. Requisitos y contratos	HCD, EDA*	Requisitos funcionales y no funcionales; contrato de frontera (de eventos versionado si EDA está activo)	Sin compuerta
F3. Diseño y arquitectura	HCD, EDA*, DevOps	Decisiones de arquitectura; especificación de <i>pipeline</i>	Sin compuerta
F4. Activos y prototipado	HCD, GenIA*	Prototipos de fidelidad creciente; activos con derechos aclarados	G3* generación
F5. Construcción	DevOps, EDA*, GenIA*	Doble cadena: código y contenido	G3* generación
F6. Verificación	DevOps, EDA*, HCD	Evidencias técnicas, de experiencia y de responsabilidad social	G1 rendimiento; G2 experiencia y responsabilidad social
F7. Despliegue, operación y evolución	DevOps	<i>Release</i> desplegado; telemetría de campo	Sin compuerta

* Elemento condicional: depende de la activación del enfoque (Sección IV-G). Las fases no se recorren en secuencia única: cada microciclo de dos semanas selecciona trabajo de las fases activas.

visual y coste. El desarrollador RA implementa la interacción, el *tracking*, el renderizado y la integración de los activos en la experiencia. El desarrollador de servicios implementa la lógica de negocio, las integraciones y, cuando EDA está activo, los productores y consumidores de eventos. El ingeniero DevOps mantiene los *pipelines*, ambientes, automatización y observabilidad. El especialista en experiencia de usuario dirige las actividades de HCD y las evaluaciones con usuarios. El responsable de calidad de integración y rendimiento ejecuta las pruebas de integración, latencia, *throughput* y resiliencia y responde por G1. El responsable de derechos y licencias, que cumple la función de *Legal Rights Management*, verifica la procedencia, licencias y derechos asociados a los activos y al contenido generado. Finalmente, el responsable de contenido prepara, mantiene y versiona los activos de augmentación y, cuando GenIA está activo, coejecuta G3 junto con el responsable de derechos y licencias. Los roles externos formales son dos: el usuario final, que participa en la definición del contexto y en G2, y el responsable de los datos, que interviene cuando GenIA está activo.

Tres roles merecen justificación explícita. El **product owner** concentra la autoridad sobre el qué: prioriza el *backlog*, decide qué entra en cada microciclo y es el único que puede aceptar un incremento como validado tras superarse G1 y G2. Su existencia separa la decisión de producto de la decisión técnica, que en su ausencia tiende a recaer en quien implementa. El **líder técnico** concentra la autoridad sobre el cómo: arbitra los compromisos entre latencia, calidad visual y coste, responde por el contrato de frontera en cualquier instancia de la metodología y decide la fase de destino cuando G1 falla; este rol absorbe la función de arquitectura de eventos cuando EDA está activo, de modo que la metodología no depende de un rol permanente asociado a un enfoque condicional. El **responsable de derechos y licencias** atiende una obligación que el dominio impone con particular fuerza y que las metodologías genéricas no contemplan: en RA el producto incorpora activos de terceros, captura entornos

donde aparecen personas y, con GenIA activo, contenido cuya procedencia debe poder acreditarse. Mantiene el registro de derechos, condiciona la entrada de cualquier activo a la cadena de contenido y coejecuta G2 y G3.

Los roles internos superan los cinco, el mismo grado que RAD [8], pero con una diferencia sustantiva: RAD centra la coordinación en identificar equipos paralelos y verificar su no-redundancia, sin detallar cómo trabajan juntos. Aquí la coordinación se ancla en dos artefactos de lectura obligatoria: el informe de evaluación con usuarios, presente en toda instancia, y el contrato de frontera entre componentes, que adopta la forma de contrato de eventos versionado cuando EDA está activo y, cuando no lo está, la de especificación de las interfaces de datos entre captura, lógica y presentación, que el dominio exige en cualquier caso [13]. Lo que la metodología fija es la existencia de un contrato explícito de frontera, no la tecnología que lo materializa.

IV-D. Artefactos

Es el eje donde los enfoques tensionan: EDA y DevOps empujan hacia formalización alta; HCD aporta artefactos deliberadamente semi-formales; GenIA añade *prompts* versionados. Se propone formalización mixta y declarada por artefacto. Son obligatoriamente formales el contrato de frontera (en su variante de eventos, mediante AsyncAPI o JSON Schema), la especificación de *pipeline*, el manifiesto de contenido de augmentación, el registro de derechos asociado a él y el informe de evaluación con métricas; se mantienen semi-formales los artefactos de exploración de la experiencia. Los *prompts* versionados se clasifican como formales: al gobernar una salida que entra al producto deben ser reproducibles y auditables. La posición es intermedia entre XP, que no recomienda artefactos no ejecutables por considerarlos inútiles, y RAD, que propone alrededor de treinta clases solo en su fase de inicialización [8]. El criterio es explícito: se formaliza lo que cruza una frontera entre equipos o entra al producto, y se deja semi-formal lo que sirve para explorar.

IV-E. *Uso recomendado*

La metodología se dirige a proyectos de RA con equipo multidisciplinario y madurez de requisitos baja, condición característica del dominio. El tamaño de referencia es el que [8] asocia a métodos con más de cinco roles diferenciados y coordinación basada en artefactos, no un umbral propio validado empíricamente. No resulta apropiada para desarrollos con requisitos cerrados y contexto conocido, donde el peso de HCD no se justifica, ni para prototipos exploratorios de un solo desarrollador, donde el coste de los contratos de frontera, las compuertas y los entregables de microciclo supera su beneficio. Dentro de ese ámbito, el carácter distribuido o de tiempo real determina la activación de EDA y la disponibilidad de datos confiables la de GenIA. Declarar estos límites es un aporte frente a los modelos comparados: RAD está clasificado como sin información específica sobre el tamaño de proyecto esperado y con mención vaga del tipo de aplicación [8].

IV-F. *Madurez*

La madurez de los enfoques es desigual: HCD y DevOps son altos por normalización y adopción industrial; EDA es medio-alto en general pero con escasa documentación específica en RA; GenIA es emergente. En consecuencia, la metodología es una primera aproximación sin difusión ni validación empírica. Su aporte en este eje no es admitir esa carencia sino operacionalizar el instrumento con que puede resolverse, en dos familias de indicadores. Los de *instrumentación* registran si el proceso se ejecuta según lo definido: frecuencia de integración y entrega a ambientes de validación, frecuencia de promociones a producción, entregables completos por microciclo, retornos disparados por cada compuerta y cobertura de contratos de frontera versionados. Los de *resultado* registran si produce mejores productos: defectos escapados a producción por incremento, retrabajo medido en trabajo rehecho tras un retorno, puntuación de usabilidad mediante SUS y latencia percibida en la augmentación. El diseño mínimo de validación es un estudio de caso múltiple en proyectos comparables, contrastando estas variables frente a un proceso ágil sin compuertas diferenciadas.

IV-G. *Flexibilidad*

Los cuatro enfoques aportan variabilidad, y la propuesta la hace explícita por enfoque. HCD y DevOps son obligatorios en toda instancia de la metodología, aunque no necesariamente estén activos en todas las fases del ciclo de vida. HCD garantiza que la comprensión del contexto y la evaluación con usuarios permanezcan presentes durante la evolución del producto, mientras que DevOps proporciona la infraestructura de automatización, integración, entrega y operación. EDA es condicional y se activa cuando el sistema distribuye estado entre varios componentes o clientes, o cuando opera bajo restricciones de tiempo real; en su ausencia la coordinación recae en el contrato de frontera. GenIA es condicional y se activa cuando existen datos confiables sobre los que apoyar la generación; su activación añade la compuerta G3 y el rol externo de responsable de los datos. Esta configurabilidad

la aleja de los modelos de procedimiento fijo: [8] recoge que Waterfall, Spiral o el modelo en V pueden considerarse procedimientos fijos, que esa fijeza dificulta adaptarlos a condiciones locales y que el Spiral en particular no sugiere ninguna variante.

IV-H. *Responsabilidad social*

La RA impone obligaciones que exceden la calidad técnica del software y que la metodología asume de forma explícita, siguiendo el principio de tratar las preocupaciones éticas como requisitos del sistema y no como una revisión posterior [30]. Estas obligaciones se agrupan en protección de las personas y del entorno, accesibilidad e inclusión, e integridad del contenido y los derechos. Su responsabilidad se distribuye principalmente entre el especialista en experiencia de usuario y el responsable de derechos y licencias, con participación del usuario final y, cuando GenIA está activo, del responsable de los datos. Su cumplimiento se verifica principalmente mediante G2 y, para contenido generado, mediante G3.

La **protección de las personas y del entorno** parte de que un sistema de RA captura el entorno de forma continua, lo que convierte la privacidad de terceros no participantes en un requisito y no en una consideración [7]; además, el usuario interactúa desplazándose en un espacio físico, de modo que la oclusión indebida del entorno relevante y la sobrecarga de atención son riesgos de seguridad física y no solo defectos de usabilidad [12]. Se incorpora el consentimiento informado en toda sesión de evaluación, coherente con el marco normativo ya adoptado [10], [15]. Responde el especialista en experiencia de usuario y se verifica en G2.

La **accesibilidad e inclusión** figura entre las líneas que la investigación reciente en RA identifica como pendientes [3]. La metodología exige declarar en F1 qué capacidades sensoriales y motoras supone la experiencia y qué alternativas ofrece a quien no las tiene; la declaración forma parte de la especificación del contexto de uso y se verifica en G2.

La **integridad del contenido y los derechos** comprende la procedencia y licencia de los activos de terceros, los derechos de imagen de las personas capturadas o representadas y, con GenIA activo, la trazabilidad del contenido generado junto con los riesgos de sesgo, información incorrecta y propiedad intelectual que los marcos de gestión de riesgo de IA identifican [17]. El responsable de derechos y licencias mantiene el registro, ningún activo entra a la cadena de contenido sin él, y la verificación ocurre en G3 para el contenido generado y en G2 para el conjunto desplegado.

IV-I. *Comparación frente a metodologías actuales*

La comparación emplea los ejes y gradaciones con que [8] clasifica Spiral, RAD y XP; esa clasificación no incluye a Scrum, por lo que queda fuera del contraste, y dado que es el marco de gestión más extendido en la industria, su caracterización bajo los mismos ejes se señala como trabajo pendiente. El Cuadro III condensa, eje por eje, la diferencia sustantiva frente a cada modelo, ya argumentada en las Secciones IV-B a IV-F.

Cuadro III
DIFERENCIA SUSTANTIVA DE LA PROPUESTA FRENTE A SPIRAL, RAD Y XP, POR EJE

Eje	Diferencia sustantiva
Ciclo	Microciclo de dos semanas, en el grado más fino de la escala (próximo a XP, por delante de RAD); a diferencia de los tres modelos, el retorno queda nominado a un rol y con fase de destino prescrita, no solo mencionado como en RAD.
Colaboración	Nueve roles internos y dos externos, más que RAD, con dos figuras de autoridad separada y un rol de derechos y licencias que ninguno de los tres modelos contempla.
Artefactos	Posición intermedia entre XP (mínima formalización) y RAD (máxima); lo distintivo no es la cantidad de artefactos sino el criterio explícito de frontera que decide cuáles se formalizan.
Uso recomendado	Declara ámbito y casos de no-uso, información que RAD no ofrece y que Spiral trata solo para proyectos grandes sin procedimiento de evaluación.
Madurez	Principal limitación de la propuesta: frente a XP, ampliamente difundido, carece de difusión y validación empírica.

V. CONCLUSIONES

El estado del arte revisado indica que la realidad aumentada carece de una metodología que trate la entrega técnica, la reacción a eventos, la validación con usuarios y la responsabilidad social como ejes de igual jerarquía y no como anexos opcionales. Frente a ese vacío, este trabajo propone una primera aproximación que articula GenIA, DevOps, EDA y HCD sobre los seis ejes de la espina de pescado de [8], sin subordinar ninguno a los demás. Cinco decisiones concentran el aporte: la estructura de cadencias anidadas, que separa la frecuencia con que se publica de la frecuencia con que se valida; el esquema de compuertas, cada una asignada a un rol y prescriptiva sobre la fase de destino del retroceso, en un punto donde Spiral y RAD no precisan qué hacer si una etapa no se valida; la definición de roles con autoridad separada, que distingue la decisión de producto del arbitraje técnico e incorpora un responsable de derechos y licencias como condición de entrada a la cadena de contenido; la dualidad de *pipelines*, que trata el contenido de augmentación como componente de primera clase junto al código; y la incorporación de la responsabilidad social como obligación verificable en compuerta, con responsable nominado, y no como declaración de intenciones.

Esta propuesta es una primera aproximación no validada: carece de difusión y de evidencia empírica, y su alcance es el de una línea base estructurada. Por eso se deja operacionalizado el instrumento de medición en dos familias de indicadores. El trabajo futuro se orienta a instrumentar la metodología en proyectos comparables de RA y contrastar las variables de resultado frente a un proceso ágil sin compuertas diferenciadas, según el estudio de caso descrito en la Sección IV-F.

REFERENCIAS

- [1] R. T. Azuma, "A survey of augmented reality," Presence: Teleoperators and Virtual Environments, vol. 6, no. 4, pp. 355–385, Aug. 1997, doi: 10.1162/pres.1997.6.4.355.
- [2] R. Azuma et al., "Recent advances in augmented reality," IEEE Comput. Graph. Appl., vol. 21, no. 6, pp. 34–47, Nov./Dec. 2001, doi: 10.1109/38.963459.
- [3] Y. Goh et al., "Wearable augmented reality: Research trends and future directions from three major venues," IEEE Trans. Vis. Comput. Graphics, 2023, doi: 10.1109/TVCG.2023.3320231.
- [4] D. Schmalstieg and T. Höllerer, Augmented Reality: Principles and Practice. Boston, MA, USA: Addison-Wesley, 2016.

- [5] J. S. Devagiri et al., "The essentials: A comprehensive survey to get started in augmented reality," IEEE Access, vol. 12, pp. 109012–109070, 2024, doi: 10.1109/ACCESS.2024.3438944.
- [6] S. Graser, F. Kirschenlohr, and S. Böhm, "User experience evaluation of augmented reality: A systematic literature review," in Proc. 17th Int. Conf. Advances in Human-oriented and Personalized Mechanisms, Technologies, and Services (CENTRIC 2024), Venice, Italy, 2024, pp. 23–40.
- [7] T. A. Syed et al., "In-depth review of augmented reality: Tracking technologies, development tools, AR displays, collaborative AR, and security concerns," Sensors, vol. 23, no. 1, Art. no. 146, 2023, doi: 10.3390/s23010146.
- [8] E. Céret, S. Dupuy-Chessa, G. Calvary, A. Front, and D. Rieu, "A taxonomy of design methods process models," Inf. Softw. Technol., vol. 55, no. 5, pp. 795–821, May 2013.
- [9] P. Milgram and F. Kishino, "A taxonomy of mixed reality visual displays," IEICE Trans. Inf. & Syst., vol. E77-D, no. 12, pp. 1321–1329, Dec. 1994.
- [10] Y.-Y. Cho, J. Sauer, and K. Choi, Augmented Reality (AR) Usability Evaluation Framework (NIST IR 8422). Gaithersburg, MD, USA: Nat. Inst. Standards Technol., 2022.
- [11] M. Sereno, X. Wang, L. Besançon, M. J. McGuffin, and T. Isenberg, "Collaborative work in augmented reality: A survey," IEEE Trans. Vis. Comput. Graphics, vol. 28, no. 6, pp. 2530–2549, Jun. 2022, doi: 10.1109/TVCG.2020.3032761.
- [12] T. C. Endsley et al., "Augmented reality design heuristics: Designing for dynamic interactions," Proc. Hum. Factors Ergon. Soc. Annu. Meet., vol. 61, no. 1, pp. 2100–2104, 2017, doi: 10.1177/1541931213602007.
- [13] ETSI, "Augmented Reality Framework (ARF); AR framework architecture," ETSI GS ARF 003 V2.1.1, 2023.
- [14] R. Roedavan, A. Pudjoatmodjo, and Y. Sugianto, "A framework for developing augmented reality applications based on the multimedia development life cycle (MDLC)," J. Informatika dan Informasi Geografi, vol. 16, no. 1, 2025, doi: 10.36982/jiig.v16i1.5183.
- [15] International Organization for Standardization, Ergonomics of human-system interaction, Part 210: Human-centred design for interactive systems, ISO 9241-210:2019, Geneva, Switzerland, 2019.
- [16] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 9459–9474.
- [17] National Institute of Standards and Technology, Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile, NIST AI 600-1. Gaithersburg, MD, USA: Nat. Inst. Standards Technol., Jul. 2024, doi: 10.6028/NIST.AI.600-1.
- [18] H. Doh, J. Shi, R. Jain, H. Kim, and K. Ramani, "An exploratory study on multi-modal generative AI in AR storytelling," arXiv:2505.15973, 2025.
- [19] L. Bass, I. Weber, and L. Zhu, DevOps: A Software Architect's Perspective. Upper Saddle River, NJ, USA: Addison-Wesley Professional, 2015.
- [20] R. Jabbari, N. bin Ali, K. Petersen, and B. Tanveer, "What is DevOps? A systematic mapping study on definitions and practices," in Proc. XP 2016 Scientific Workshops, Edinburgh, U.K., May 2016, Art. no. 12.
- [21] H. Cabane and K. Farias, "On the impact of event-driven architecture on performance: An exploratory study," Future Gener. Comput. Syst., vol. 153, pp. 52–69, 2024.

- [22] AsyncAPI Initiative, AsyncAPI Specification, Version 3.0.0, 2023. [Online]. Available: <https://www.asyncapi.com/docs/reference/specification/v3.0.0>
- [23] D. Vaquero-Melchor, A. M. Bernardos, and L. Bergesio, "SARA: A microservice-based architecture for cross-platform collaborative augmented reality," *Appl. Sci.*, vol. 10, no. 6, Art. no. 2074, Mar. 2020.
- [24] A. Seffah, J. Gulliksen, and M. C. Desmarais, Eds., *Human-Centered Software Engineering: Integrating Usability in the Software Development Lifecycle*. Dordrecht, Netherlands: Springer, 2005.
- [25] X. Ferre, N. Juristo, and A. Moreno, "Which, when and how usability techniques and activities should be integrated," in *Human-Centered Software Engineering*, A. Seffah, J. Gulliksen, and M. C. Desmarais, Eds. Dordrecht, Netherlands: Springer, 2005, pp. 173–200.
- [26] J. Nielsen, "Heuristic evaluation," in *Usability Inspection Methods*, J. Nielsen and R. L. Mack, Eds. New York, NY, USA: Wiley, 1994, pp. 25–62.
- [27] F. Paternò, *Model-Based Design and Evaluation of Interactive Applications*, 1st ed. London, U.K.: Springer, 2000.
- [28] A. Picardi and G. Caruso, "User-centered evaluation framework to support the interaction design for augmented reality applications," *Multimodal Technol. Interact.*, vol. 8, no. 5, Art. no. 41, 2024.
- [29] B. Henderson-Sellers and J. Ralyté, "Situational method engineering: State-of-the-art review," *J. Univers. Comput. Sci.*, vol. 16, no. 3, pp. 424–478, 2010.
- [30] IEEE, IEEE Standard Model Process for Addressing Ethical Concerns during System Design, IEEE Std 7000-2021, 2021, doi: 10.1109/IEEESTD.2021.9536679.